MODELADO SEMÁNTICO DE POWER BI CON GITHUB COPILOT Y MCP

Tema 4

Ingeniería de instrucciones para el modelado



Pedir bien, permitir poco y comprobar siempre

Agenda

1. **Pedir bien**
   Los cinco componentes de un buen prompt
2. **Primero el plan**
   Proponer antes de ejecutar y limitar el alcance
3. **Lo que se pide y lo que se permite**
   Instrucciones, prompts reutilizables y herramientas
4. **Revisar antes de aceptar**
   Aprobaciones, pasos y cambios en archivos
5. **Errores frecuentes**
   Y su antídoto



Al terminar este tema

- Redactar prompts **verificables** para tareas de modelado
- Configurar **instrucciones del repositorio** y **prompts reutilizables**
- Limitar al agente con las **herramientas**, no solo con palabras

Los cinco componentes de un buen prompt

Lo que ustedes no definen, lo decide el agente



1. Contexto

Qué modelo, qué tablas, qué situación de negocio



2. Objetivo

Qué resultado concreto esperan



3. Restricciones

Qué no se toca y qué no se hace



4. Formato

Tabla, plan numerado, solo el DAX…



5. Criterio

Cómo sabrán que quedó bien

El mismo pedido, dos resultados

«Revisa las relaciones y mejóralas»

- El agente decide **qué significa mejorar**
- Elige **qué herramientas** y **qué servidor** usar
- Puede **cambiar el modelo o editar archivos** sin plan
- La respuesta es difícil de **comprobar**

Contexto + objetivo + restricciones + formato + criterio

- Alcance claro: **solo evaluar**, no cambiar
- Salida en **tabla**, con las columnas pedidas
- Un **criterio** que se verifica: «deben aparecer todas las relaciones»
- **Ningún cambio** no pedido

Primero el plan, después la ejecución



«No ejecutes nada. Propón un plan numerado y espera mi aprobación.»

- **Corregir un plan es gratis**; deshacer un cambio en un modelo publicado, no
- Un buen plan dice en cada paso: herramienta, objeto, riesgo y cómo se revierte
- **Limiten el alcance:** «solo la tabla de ventas», «no borres», «máximo 5 filas»
- Dividan las tareas grandes en **lotes** pequeños y verificables

Lo que se pide y lo que se permite



Instrucciones del repositorio

.github/copilot-instructions.md. Reglas que el agente lee en cada mensaje: idioma, convenciones, «plan antes de cambiar». Una petición permanente.



Prompts reutilizables

.github/prompts/*.prompt.md. Un buen prompt guardado; se lanza con /nombre en el chat. Una petición repetible.



Herramientas permitidas

Configure Tools en el chat: qué herramientas y qué servidores MCP tiene el agente. No es una petición: es una cerradura.

Instrucciones del repositorio: qué incluir

MARKDOWN · .github/copilot-instructions.md (plantilla)

```
## Contexto
- Modelo semántico <nombre>: tablas de hechos <…>,
  dimensiones <…>. Usa los nombres exactos.

## Respuestas
- Responde en español, con tablas en Markdown.
- Cuando des un número, muestra la consulta DAX.

## Seguridad
- Antes de cualquier cambio, propone un plan
  numerado y espera mi aprobación.
- Antes de borrar o renombrar, busca qué lo usa:
  otras medidas y los visuales del informe.

## Convenciones
- Divide con DIVIDE; inteligencia de tiempo sobre
  la tabla de fechas; formatos de moneda y %.
```

- Se escribe **una vez** y aplica a **todos los chats** del repositorio
- Reglas **concretas y comprobables**
- Corto: **25 a 40 líneas**; el detalle va a los prompts
- Se versiona con el proyecto: es **del equipo**, no de una persona

Un prompt reutilizable

MARKDOWN · .github/prompts/plan-de-cambio.prompt.md

```
---
agent: 'agent'
description: 'Plan de cambio en el modelo, sin ejecutar nada'
argument-hint: 'Describa el cambio que necesita'
---
Necesito este cambio en el modelo conectado:
${input:cambio}

NO EJECUTES NADA. Solo propone un plan.
1. Lista lo que depende del objeto: otras medidas
   y visuales del informe.
2. Plan numerado: herramienta, objeto, riesgo y
   cómo se revierte.
3. Cómo comprobarías que quedó bien.
```

- Encabezado: **descripción**, **pista** y tipo de agente
- **${input:…}** recibe lo que se escribe tras el comando
- Se lanza con **/plan-de-cambio** en el chat
- Opcional: **tools** limita las herramientas de ese prompt

Revisar antes de aceptar



La aprobación dice qué herramienta y qué operación

Lean el Input: List y Get leen; Create, Update y Delete escriben. Verifiquen también de qué servidor viene.



Allow Once o Allow in this Session

La aprobación por sesión vale para toda la herramienta, también para sus escrituras. Para lo que escribe, Allow Once.



Keep o Undo en los archivos

Si el agente edita archivos, VS Code muestra «N files changed» con Keep y Undo. Lo que no pidieron, Undo.



Lo que hizo queda en los pasos

Debajo de la respuesta, «Completed … steps»: cada herramienta y con qué datos la llamó.

Errores frecuentes y su antídoto

Error	Qué pasa	Antídoto
«Arregla el modelo»	El agente decide qué tocar	Objetivo concreto y restricciones
Pedir todo a la vez	Respuestas largas, errores escondidos	Un lote por mensaje; plan primero
Confiar solo en la instrucción	La regla se ignora con algunos modelos	Quitar la herramienta en Configure Tools
Aceptar sin leer	Keep a cambios que no pidieron	Leer la operación y los archivos cambiados
Creer el número	Cifras sin consulta detrás	Pedir la consulta DAX y comparar
Pegar datos sensibles	Salen del equipo hacia el servicio	Nombres de objetos, no datos de clientes

Con planes de cuota limitada, un buen prompt también ahorra mensajes

DEMOSTRACIÓN

Del prompt vago a la biblioteca

Ver el riesgo de un prompt vago y cómo lo cierran las instrucciones, los prompts y las herramientas

1. Prompt vago: el agente edita archivos → Undo
2. Prompt estructurado: respuesta verificable, sin cambios
3. Configure Tools: quitar edición, terminal y el servidor que escribe
4. copilot-instructions.md: mismo pedido corto, mejor respuesta
5. /plan-de-cambio: borrar una medida sin romper el informe

Modelo del curso · 5 mensajes

LABORATORIO

Laboratorio 4: Biblioteca de prompts

Construir y probar las instrucciones y los prompts del proyecto, sin cambiar el modelo

1. Arranque: chat Local y Configure Tools
2. Prompt vago contra prompt estructurado
3. Su copilot-instructions.md, y probarlo
4. Tres archivos .prompt.md
5. Probar, refinar y comparar con la biblioteca base

60 min · unos 7 mensajes de Copilot

Ideas clave

- Un buen prompt tiene contexto, objetivo, restricciones, formato y criterio
- Primero el plan: corregir un plan es gratis
- Instrucciones y prompts son peticiones; las herramientas son la cerradura
- Un servidor de solo lectura no protege los archivos del proyecto: también hay que quitar la edición
- En cada aprobación, lean la operación; a lo que no pidieron, Skip o Undo
- Las instrucciones y los prompts del repositorio son un activo del equipo

Siguiente: Tema 5 · Creación y modificación del modelo